InvenTree Frontend Developer Guide

Scope: this guide covers **only** `src/frontend/` (the InvenTree SPA). It intentionally does not reference `src/backend/` or anything else in the repo. All file paths below are relative to `src/frontend/` unless stated otherwise. The project actually has two parallel source roots inside `src/frontend/`:

- **`src/`** — the application itself.
- **`lib/`** — a separately publishable package, `@inventreedb/ui`, built from `lib/` via `tsconfig.lib.json` / `vite.lib.config.ts` (see `package.json`'s `"lib"` script). It is the **plugin SDK surface**: the stable set of hooks/components/types (`InvenTreeTable`, `ApiEndpoints`, `ModelType`, `TableColumn`, `ApiFormFieldSet`, `RowActions`, etc.) that third-party InvenTree plugins import to build custom panels/forms/tables. `src/` imports from it via the `@lib/*` path alias (`tsconfig.json`: `"paths": { "@lib/*": ["./lib/*"] }`).

This distinction matters: several "core" primitives (the API endpoint enum, `apiUrl()`, the table hook, row-action helpers) live in `lib/`, not `src/`, because they need to be usable both internally and by external plugins.

---

1. Overall Architecture

```
src/frontend/
├── src/
│   ├── App.tsx              # axios instance, QueryClient, trace-id headers
│   ├── main.tsx              # app entry, global CSS imports
│   ├── router.tsx             # react-router-dom route tree
│   ├── views/                 # top-level app shells (Desktop/Mobile/Main)
│   ├── pages/                 # route-bound page components, by domain
│   ├── components/            # reusable UI, organized by concern (not domain)
│   ├── tables/                 # per-domain table components + filters
│   ├── forms/                  # per-domain ApiFormFieldSet builders
│   ├── hooks/                  # app-level hooks (UseForm, UseInstance, UseModal.
│   ├── functions/              # auth, api error handling, notifications, forms h
│   ├── states/                 # Zustand global stores (User, LocalState, ServerA
│   ├── contexts/               # React Context providers (Api, Theme, Language)
│   ├── styles/, main.css.ts, theme.ts  # vanilla-extract styling
│   └── defaults/                # static config (nav links, defaults)
└── lib/
    ├── components/, hooks/, functions/, states/, types/, enums/, plugin/
    └── index.ts                # public barrel export (@inventreedb/ui)
```

Why this split exists: `src/components/` is organized by *UI concern* (barcodes, buttons,

dashboard, details, nav, panels, tables…) while `tables/`, `forms/`, and `pages/` are organized *by backend domain* (`part`, `stock`, `build`, `company`, `purchasing`, `sales`, `settings`, `machine`…), mirroring the Django app names on the backend. This means: if you're building a new generic widget, it goes in `components/<concern>/`; if you're building a table/form/page for a specific data model, it goes in `tables|forms|pages/<domain>/`.

Data flow, end to end: Zustand stores (`states/`) hold session/user/settings state → React Query (`useQuery`) fetches data through the shared axios instance (`App.tsx`'s `api`) using URLs built from the `ApiEndpoints` enum → components render that data via shared primitives (`InvenTreeTable`, `DetailsTable`, `ApiForm`) → mutations go back through the same axios instance as plain `api({...})` calls (not `useMutation`) → success handlers manually refresh the affected query (bump a `tableKey`, or refetch an instance) rather than relying on cache invalidation.

---

2. API Layer

2.1 The axios instance

File: `src/App.tsx`

```
export const api = axios.create({});

export function setApiDefaults() {
  const { getHost } = useLocalState.getState();
  api.defaults.baseURL = getHost();
  api.defaults.timeout = 5000;
  api.defaults.withCredentials = true;
  api.defaults.withXSRFToken = true;
  api.defaults.xsrfCookieName = 'csrftoken';
  api.defaults.xsrfHeaderName = 'X-CSRFToken';
}
```

There is **one global axios instance**, created once, configured (baseURL etc.) whenever auth state changes (login/logout — see `states/UserState.tsx`). Auth is **Django session cookie + CSRF**, not a bearer token: `withCredentials: true` plus the CSRF double-submit-cookie scheme means the browser sends `sessionid` / `csrftoken` cookies automatically and axios reads the `csrftoken` cookie itself to set the `X-CSRFToken` header on unsafe methods. You will not find a manual `Authorization: Bearer ...` header anywhere in normal request code.

The React Query `QueryClient` is also constructed in `App.tsx`, with one global default: `refetchOnWindowFocus: false`.

It's exposed to the component tree via a small custom context rather than the library's

it's exposed to the component tree via a small custom context rather than the library's own provider directly — **`src/contexts/ApiContext.tsx`**:

```
const ApiContext = createContext<AxiosInstance | null>(null);
export const ApiProvider = ({ api, client, children }) => (
  <QueryClientProvider client={client}>
    <ApiContext.Provider value={api}>{children}</ApiContext.Provider>
  </QueryClientProvider>
);
export const useApi = () => useContext(ApiContext)!; // throws if missing
```

Wired up once, at the app root, in `src/views/DesktopAppView.tsx`.

2.2 Endpoint enumeration — never hardcode URLs

File: `lib/enums/ApiEndpoints.tsx` — one big string enum listing every backend route:

```
export enum ApiEndpoints {
  user_list = 'user/',
  user_me = 'user/me/',
  auth_login = 'auth/v1/auth/login',
  part_list = 'part/',
  ...
}
```

File: `lib/functions/Api.tsx` — the single function that turns an enum member into a real URL:

```
export function apiUrl(endpoint: ApiEndpoints | string, pk?: any, pathParams?: Pat
  let _url = endpoint;
  if (!_url.startsWith('/')) _url = apiPrefix() + _url; // prefixes '/api/'
  if (_url && pk) {
    if (_url.indexOf(':id') >= 0) _url = _url.replace(':id', `${pk}`);
    else _url += `${pk}/`;
  }
  if (_url && pathParams) {
    for (const key in pathParams) _url = _url.replace(`:${key}`, `${pathParams[key
  }
  return _url;
}
```

Rule: every API call in the app goes through `apiUrl(ApiEndpoints.xxx, pk?, pathParams?)`. Never write a literal `/api/part/1/` string in a component.

2.3 Fetching data — React Query, no bespoke per-endpoint hooks

`@tanstack/react-query`'s `useQuery` is used everywhere for reads. There is no code

generation from the OpenAPI/DRF schema — model types are hand-written TypeScript interfaces (`lib/types/*.tsx`), and field metadata (required/read-only/choices/labels) is instead fetched **at runtime** via HTTP `OPTIONS` requests, not compile-time codegen.

Single-instance fetch — `src/hooks/UseInstance.tsx` (`useInstance({ endpoint, pk })`):

```
const instanceQuery = useQuery<T>({
  queryKey: ['instance', endpoint, pk, paramsKey, disabled],
  retry: (failureCount, error: any) => error.response?.status == 404 ? false : fai
  queryFn: async () => {
    const url = apiUrl(endpoint, pk, pathParams);
    return api.get(url, { timeout: 10000, params }).then((r) => { setInstance(r.da
  },
});
```

Used on essentially every "detail" page (part, stock item, order...).

List/table fetch — `src/components/tables/InvenTreeTable.tsx` (used by every table — see §3):

```
const { data: apiData, isFetching, isLoading, refetch } = useQuery({
  queryKey: ['tabledata', url, tableState.page, props.params, sortStatus.columnAcc
             sortStatus.direction, tableState.tableKey, tableState.filterSet.activ
  retry: 5,
  retryDelay: (attempt) => (1 + attempt) * 250,
  throwOnError: (error) => { showApiErrorMessage({ error, title: t`Error loading t
  enabled: !!url && !tableData,
  queryFn: fetchTableData, // api.get(url, { params: queryParams }), extracts resp
});
```

2.4 Mutations — plain axios, not `useMutation`

`useMutation` is not used anywhere in this codebase. Creates/updates/deletes go through the shared `api` instance directly, inside the generic `ApiForm` component (`src/components/forms/ApiForm.tsx`):

```
return api({
  method: method,                // 'post' | 'put' | 'patch' | 'delete'
  url: url,                       // apiUrl(props.url, props.pk)
  data: hasFiles ? formData : jsonData,
  headers: { 'Content-Type': hasFiles ? 'multipart/form-data' : 'application/json'
}).then((response) => {
  if ([200, 201, 204].includes(response.status)) {
    props.onFormSuccess?.(response.data, form);
    if (props.follow && props.modelType && followPk) navigate(getDetailUrl(props.m
    else if (props.table) props.table.refreshTable();
  }
```

```
});
```

Cache "invalidation" is done by changing the query key, not `queryClient.invalidateQueries()`. `lib/hooks/UseTable.tsx`'s `refreshTable()` regenerates a random `tableKey`, which is part of the table's `useQuery` key (§2.3), forcing a refetch. This is the idiom to know: after any mutation that should refresh a list, call `table.refreshTable()` — don't reach for React Query's invalidation API.

2.5 Error handling

No axios interceptors exist. Error handling is per-call-site plus two shared helpers:

- `src/functions/api.tsx` → `extractErrorMessage({ error, field, defaultMessage })` — pulls a message out of `error.response.data` (checking `field` / `non_field_errors`) or falls back to a status-code keyed default message.
- `src/functions/notifications.tsx` → `showApiErrorMessage()` — calls `extractErrorMessage()` then shows a red Mantine notification. Used from table/query `throwOnError` callbacks.
- `lib/functions/Notification.tsx` → generic helpers: `permissionDenied()`, `invalidResponse(code)`, `showTimeoutNotification()`.
- Form-specific 400 errors are walked recursively in `ApiForm.tsx` and mapped onto individual fields via React Hook Form's `form.setError(path, { message })` — see §5.4.

2.6 Loading state

Loading state is React Query's own `isLoading` / `isFetching`, sometimes mirrored into local/table state (e.g. `InvenTreeTable` combines its data query and its OPTIONS-metadata query into one `tableState.isLoading` flag that drives the mantine-datatable spinner overlay).

2.7 Caching

- Global default: `refetchOnWindowFocus: false` (`App.tsx`).
- Per-query overrides are common: short `gcTime` for OPTIONS metadata (rarely changes), `retry: 5` with linear backoff for table data, `retry` that gives up immediately on 404 for instance fetches.
- A **separate, non-React-Query cache** persists translated column names/sort/page-size/hidden columns across sessions: `lib/states/StoredTableState.tsx`.

2.8 How to Add a New API Call — Checklist

1. **Add the endpoint** to `lib/enums/ApiEndpoints.tsx` (relative path, no `/api/` prefix).
2. **Build the URL** with `apiUrl(ApiEndpoints.your_endpoint, pk?, pathParams?)` — never hardcode.

3. **Reads**: wrap in `useQuery` (either reuse `useInstance()` for a single object, or write a small local `useQuery` if it's a one-off — follow the `queryKey` pattern of including every param that should trigger a refetch).
4. **Writes**: prefer going through `ApiForm` / `useCreateApiFormModal` / `useEditApiFormModal` / `useDeleteApiFormModal` (§5) rather than a hand-rolled `api.post(...)` — you get validation, error mapping, and success notifications for free. Only write a raw `api.post/patch/delete(...)` call for actions that aren't really "forms" (e.g. a one-click status-change button).
5. **Error handling**: for reads, add `throwOnError` + `showApiErrorMessage()`; for writes via `ApiForm`, this is automatic.
6. **Refresh affected lists**: call `table.refreshTable()` (if a table should reflect the change) or `refreshInstance()` (if a detail page should re-fetch).
7. **Loading state**: use the query's `isLoading` / `isFetching` directly in JSX — don't invent a separate `useState` for it unless combining multiple queries.
8. **Types**: add/extend a hand-written interface in `lib/types/` if the response shape is new; there's no codegen step to run.

---

3. Tables

Tables are the single most important reusable subsystem in this frontend — nearly every list view in the app is a thin, declarative wrapper around one shared component.

3.1 Core component

File: `src/components/tables/InvenTreeTable.tsx` (~1000 lines), built on **`mantine-datatable`** (not mantine-react-table, not a custom grid), with data fetching via `useQuery` and row context menus via `mantine-contextmenu`.

Two exports: `InvenTreeTableInternal<T>` (the real implementation — exported "raw" so it can also be handed to plugins) and `InvenTreeTable<T>` (the app-facing wrapper that injects `api`, `navigate`, `showContextMenu`, and URL search params, then forwards to `InvenTreeTableInternal`).

Props (`InvenTreePropsTable<T>`, `lib/types/Tables.tsx:192-226`): `params`, `defaultSortColumn`, `enableBulkDelete`, `enableFilters`, `enableSelection`, `enableSearch`, `enablePagination`, `enableColumnSwitching`, `tableFilters`, `tableActions`, `rowActions`, `onRowClick`, `modelType` (drives navigate-to-detail-on-click), `height`, `noHeader`, and more.

3.2 Column definitions

Type: `lib/types/Tables.tsx` — `TableColumn<T> = { accessor: string } & TableColumnProps<T>` where `TableColumnProps` includes `title`, `sortable`,

`TableColumnProps<T>`, where `TableColumnProps` includes `title`, `sortable`, `switchable`, `hidden`, `defaultVisible`, `render`, `filter`, `width`, `copyable`, etc.

Columns are a plain array, usually built by a small function and memoized. Real example — `src/tables/part/PartTable.tsx`:

```
function partTableColumns(): TableColumn[] {
  return [
    PartColumn({ part: '', accessor: 'name', filter: ['active', 'locked', 'starred
    IPNColumn({ accessor: 'IPN' }),
    { accessor: 'revision', sortable: true },
    DescriptionColumn({}),
    CategoryColumn({ accessor: 'category_detail' }),
    { accessor: 'total_in_stock', sortable: true, render: (record) => { /* custom
  ];
}
```

Note the mix of raw column objects and **shared column-factory helpers** — `src/components/tables/ColumnRenderers.tsx` exports `PartColumn`, `IPNColumn`, `DescriptionColumn`, `BooleanColumn`, `StatusColumn`, `LocationColumn`, `UserColumn`, `LinkColumn`, etc. — reused across dozens of tables. **Prefer these over writing a custom render function** when the column is a common type (a linked model, a status badge, a boolean, a user).

3.3 Data fetching, sorting, filtering, pagination — all inherited automatically

A table only supplies a `url` (its list endpoint) and `params` (static filters); `InvenTreeTable` handles the rest:

- **Fetching**: `api.get(url, { params: queryParams })`, extracting `response.data.results` / `count` (DRF pagination shape).
- **Sorting**: server-side. Setting `sortable: true` on a column is enough; `ordering` on the column overrides the backend field name if it differs from `accessor`. Sort state persists per-table via `useStoredTableState`.
- **Searching**: `enableSearch` renders a search box that sets `tableState.searchTerm`, sent as a `search=` query param.
- **Filtering**: declare a `TableFilter[]` (type in `lib/types/Filters.tsx`) in a sibling `*TableFilters.tsx` file, e.g. `src/tables/part/PartTableFilters.tsx`:

  ```
  { name: 'active', label: t`Active`, description: t`Filter by part active statu
  ```

  Pass it as `tableFilters` in `props`. The funnel-icon drawer UI is `src/components/tables/FilterSelectDrawer.tsx`. Columns can also carry an inline filter icon via `col.filter`.
- **Pagination**: server-side, offset/limit. Page sizes: `[10, 15, 20, 25, 50, 100, 500]`. Fully wired into mantine-datatable's built-in pagination footer — nothing to implement per-

table.

3.4 Row actions

Type: `RowAction` (`lib/types/Tables.tsx`) — `{ title, tooltip, color, icon, onClick, hidden, disabled }`. **Component:** `lib/components/RowActions.tsx`, with preset factories: `RowViewAction`, `RowEditAction`, `RowDuplicateAction`, `RowDeleteAction`, `RowCancelAction`.

```
const rowActions = useCallback((record: any): RowAction[] => {
  const can_edit = user.hasChangePermission(ModelType.part);
  return [
    RowEditAction({ hidden: !can_edit, onClick: () => { setSelectedPart(record); e
    RowDuplicateAction({ hidden: !user.hasAddPermission(ModelType.part), onClick:
  ];
}, [user, editPart]);
```

Pass this as `rowActions` in `props` and `InvenTreeTable` appends the actions column automatically (also surfaced on right-click via `mantine-contextmenu`).

Toolbar-level (non-per-row) bulk actions use a different component, **`src/components/items/ActionDropdown.tsx`** (presets: `EditItemAction`, `DeleteItemAction`, `DuplicateItemAction`), passed as `tableActions` in `props`.

3.5 Custom cell rendering

Any column can supply `render: (record, index?) => ReactNode`. For common cases, use `ColumnRenderers.tsx` (e.g. `RenderPartColumn` shows a thumbnail + status icons; `TableStatusRenderer` — used by `StatusColumn` — renders a colored Mantine `Badge`). Write a custom inline `render` only for genuinely one-off logic (e.g. `PartTable.tsx`'s hover-card stock breakdown). `copyable: true` on a column auto-adds a copy-to-clipboard hover button — don't hand-roll this.

3.6 How to Add a New Table — Step by Step

1. **Pick/confirm the API list endpoint** exists in `ApiEndpoints` (§2.2).
2. Create `src/tables/<domain>/<Model>Table.tsx`.
3. Write a `columns()` function (memoized) returning `TableColumn[]`, reusing `ColumnRenderers.tsx` factories wherever the column type is common (linked model, status, boolean, description…).
4. If the list needs filters, create a sibling `<Model>TableFilters.tsx` exporting a `TableFilter[]`-returning function.
5. In the table component: call `useTable('unique-table-name')` (`lib/hooks/UseTable.tsx`) to get a `TableState`.

6. If create/edit/delete is needed, wire up `useCreateApiFormModal` / `useEditApiFormModal` / `useDeleteApiFormModal` (§5.5) with `table: tableState` so they auto-refresh the table on success.
7. Build a `rowActions` callback (§3.4) gated by `user.hasChangePermission/hasAddPermission(...)`.
8. Render:

```
<InvenTreeTable
  url={apiUrl(ApiEndpoints.your_model_list)}
  tableState={tableState}
  columns={columns}
  props={{ tableFilters, tableActions, rowActions, enableSelection: true, para
/>
```

9. Drop the component into the relevant page/panel.

That's the entire minimal surface — fetch, sort, filter, search, paginate, column-visibility toggling and caching all come for free from `InvenTreeTable`. **Do not reimplement any of these.**

---

4. Cards

There is no single generic `<Card>` wrapper reused everywhere. Instead, the project consistently uses **Mantine's `Paper`** (`withBorder`, small padding) as the card primitive, plus two purpose-built composite components.

4.1 Building blocks

- **`ItemDetailsGrid`** (`src/components/details/ItemDetails.tsx`) — a `Paper` wrapping a responsive `SimpleGrid` (`cols={{ base: 1, '900px': 2 }}`) — the standard 1→2 column layout for stacking detail cards.
- **`DetailsTable`** (`src/components/details/Details.tsx`) — the actual "card": a titled key/value table.

```
<Paper p='xs' withBorder>
  <Stack gap='xs'>
    {title && <StylishText size='lg'>{title}</StylishText>}
    <Table striped>
      <Table.Tbody>{visibleFields.map((f, i) => <DetailsTableField field={f} i
    </Table>
  </Stack>
</Paper>
```

- **`DashboardWidget`** (`src/components/dashboard/DashboardWidget.tsx`) — the dashboard's card wrapper: `<Paper withBorder shadow='sm' p='xs'>` around a widget's

own `render()` output.

- **`PanelGroup`** (`src/components/panels/PanelGroup.tsx`) — the tabbed detail-page container that hosts cards like `DetailsTable`, `AttachmentPanel`, `NotesPanel` via a `PanelType[]` config.

4.2 Data binding

Cards don't fetch their own data (except dashboard widgets, which self-fetch). A detail page fetches its instance (`useInstance`), builds a `DetailsField[]` array with `useMemo`, and passes `{ item, fields }` straight into `DetailsTable`:

```
<ItemDetailsGrid>
  <DetailsTable fields={topLeftFields} item={data} />
  <DetailsTable fields={topRightFields} item={data} />
</ItemDetailsGrid>
```

`DetailsTableField` reads each value via `getValueAtPath(item, field.name)` and picks a renderer (`BooleanValue`, `DateValue`, `StatusValue`…) based on `field.type`.

4.3 Styling conventions

`Paper p='xs' withBorder` for flat detail-page cards; add `shadow='sm'` / `shadow='xs'` for elevated dashboard cards. Card titles use the shared `StylishText` component. Internal spacing: `Stack gap='xs'`.

4.4 How to Create a New Card

1. Decide: is this a **detail-page field group** or a **dashboard widget**?
2. **Detail-page card**: build a `DetailsField[]` array (see existing examples in any `pages/<domain>/<Model>Detail.tsx`), and render `<DetailsTable fields={...} item={data} />` inside an `<ItemDetailsGrid>` alongside any sibling cards.
3. **Dashboard widget**: add an entry to `src/components/dashboard/DashboardWidgetLibrary.tsx` as `{ label, title, description, render: () => <YourWidget /> }`; `DashboardWidget` supplies the `Paper` chrome automatically.
4. For anything else genuinely card-shaped and one-off, just use `<Paper p='xs' withBorder><Stack gap='xs'>...</Stack></Paper>` directly — don't invent a new wrapper component.

---

5. Forms

5.1 Architecture

The form engine is **`react-hook-form`**, wrapped by a custom, API-schema-driven component: **`src/components/forms/ApiForm.tsx`** (Mantine's own form hook is only used for a couple of unrelated non-API forms, like `AuthenticationForm.tsx`).

Two layers:

- **`OptionsApiForm`** — fetches the DRF `OPTIONS` schema for the target URL via `useQuery`, merges it with the caller's declared fields, then renders `ApiForm`.
- **`ApiForm`** — the actual form: renders one `ApiFormField` per field, manages `react-hook-form` state, builds the request payload, issues the HTTP call.

Convenience wrappers `CreateApiForm` / `EditApiForm` / `DeleteApiForm` just force the HTTP method.

5.2 Declarative field schema

Type: `ApiFormFieldSet = Record<string, ApiFormFieldType>` (`lib/types/Forms.tsx`), where a field can override `label`, `field_type`, `required`, `hidden`, `disabled`, `filters` (for related-field queries), `default`, `onValueChange`, etc.

Real example — `src/forms/PartForms.tsx`:

```
const fields: ApiFormFieldSet = {
  category: { filters: { structural: false } },
  name: {},
  IPN: {},
  tags: TagsField({}),
  purchaseable: { value: purchaseable, onValueChange: (v) => setPurchaseable(v) },
};
```

Most fields are just `{}`. Label, type, required-ness, choices, and help text are filled in automatically from the live API schema (§5.3) — you only specify overrides and behavior hooks.

5.3 Submission — schema introspection via `OPTIONS`

`OptionsApiForm` issues `api.options(url)`, and `extractAvailableFields()` (`src/functions/forms.tsx`) reads `response.data.actions[METHOD]` (PATCH reuses the PUT schema) to build each field's `field_type`, `description` (from `help_text`), `value`/`default`, and `disabled` (from `read_only`). This is merged with your local field overrides (`constructField()`), including nested/dependent fields.

On submit, `ApiForm` builds `FormData`/JSON from the form values and calls:

```
api({ method, url, data: hasFiles ? formData : jsonData, headers: {...} });
```

So: **Create → POST**, **Edit → PATCH (with `fetchInitialData: true` to GET existing values first)**, **Delete → DELETE** — all against the same endpoint, with the field set and permission gating derived live from `OPTIONS` for that method (a 403/missing-method response triggers `permissionDenied()`).

5.4 Validation & error handling

- **Client-side**: `react-hook-form`'s `formState.isValid` gates the submit button; `required` comes from the server schema unless overridden.
- **Server-side (400 responses)**: `ApiForm`'s catch block recursively walks `error.response.data` and calls `form.setError(path, { message })` per field (including dotted/indexed paths for nested/table fields). Errors for unknown/hidden fields or `non_field_errors` / `__all__` are collected into a `nonFieldErrors` state shown in a top-level `Alert`.
- **Non-400 errors**: `invalidResponse(status)` / `showTimeoutNotification()` (network errors). `props.onFormError?.(error, form)` is always called as an escape hatch.
- **Success**: green notification via `props.successMessage` (defaults like `t\`Item Created\`` come from the modal hooks below); modal auto-closes unless "keep open" is toggled; optionally navigates to the new object's detail page (`props.follow` + modelType) or calls table.refreshTable() / props.onFormSuccess.

5.5 Modal hooks — the normal way to open a form

File: `src/hooks/UseForm.tsx` — `useCreateApiFormModal`, `useEditApiFormModal`, `useDeleteApiFormModal`, `useBulkEditApiFormModal`, all built on `useApiFormModal()` (defaults: Create→POST + "Item Created", Edit→PATCH + fetch-initial-data + "Item Updated", Delete→DELETE + red confirm button).

Real usage — `src/pages/part/PartDetail.tsx`:

```
const editPart = useEditApiFormModal({
  url: ApiEndpoints.part_list, pk: part.pk, title: t`Edit Part`,
  fields: usePartFields({ create: false, partId: part.pk }),
  onFormSuccess: refreshInstance,
});
const deletePart = useDeleteApiFormModal({
  url: ApiEndpoints.part_list, pk: part.pk, title: t`Delete Part`,
  onFormSuccess: () => navigate(...),
});
// elsewhere: <Button onClick={() => editPart.open()}>Edit</Button>
// and render editPart.modal somewhere in the tree
```

When paired with a table (`table: tableState` in the hook props), success auto-refreshes that table.

5.6 Reusable input components

`src/components/forms/fields/`: `RelatedModelField` (async searchable related-object picker, with an inline "create new" that opens a nested `useCreateApiFormModal`), `TreeField` (category/location pickers), `IconField`, `DateField` / `DateTimeField`, `TagsField`, `TableField` (editable line-item sub-table, e.g. BOM/order lines), `NestedObjectField` / `DependentField`, plus plain `ChoiceField` / `BooleanField` / `NumberField` / `TextField`. Reusable field-schema builders (not components) live in `src/forms/CommonFields.tsx` (e.g. `TagsField()`, `ProjectCodeField()`).

5.7 How to Build a New Form

1. Decide the target endpoint + HTTP method (usually reusing an existing model's `_list` endpoint with a `pk` for edit/delete).
2. Create (or extend) a `<Domain>Forms.tsx` file in `src/forms/` exporting a hook that returns an `ApiFormFieldSet` — start every field as `{}` and only add overrides you actually need (`filters`, `default`, `onValueChange`, `hidden`, `field_type`).
3. Use `useCreateApiFormModal` / `useEditApiFormModal` / `useDeleteApiFormModal` from `src/hooks/UseForm.tsx` — don't build a custom modal + manual axios call.
4. Pass `table: tableState` if a table should auto-refresh on success, or `onFormSuccess` for custom behavior (navigate, refetch instance).
5. Trigger it from a button/menu item via `.open()`; render `.modal` once in the tree.
6. Let server-side `OPTIONS` schema drive labels/required/choices — don't duplicate that metadata by hand unless overriding it.

---

6. Authentication

Auth is centered in **`src/functions/auth.tsx`**, with global state in Zustand stores under **`src/states/`**.

6.1 Session model

Cookie-based (Django session + CSRF), not a stored token. No `localStorage` / `sessionStorage` auth token exists anywhere. `App.tsx` sets `withCredentials: true` and CSRF cookie/header names; `getCsrfCookie()` / `clearCsrfCookie()` (`functions/auth.tsx`) read/clear the `csrftoken` cookie directly from `document.cookie`. `states/UserState.tsx`'s `fetchUserToken()` first checks whether `csrftoken` / `sessionid` cookies exist, then confirms authentication by hitting the session endpoint (`ApiEndpoints.auth_session`) and checking `response.data.meta.is_authenticated`.

The only thing Zustand persists to `localStorage` (key `session_settings`, via

The only thing Zustand persists to `localStorage` (key `session-settings`, via `LocalState.tsx`'s `persist()`) is non-sensitive UI state (host, theme, language, dashboard layout) — never credentials.

6.2 Login flow

`pages/Auth/Login.tsx` renders `AuthenticationForm`. The actual call, `doBasicLogin()` (`functions/auth.tsx`), posts to `ApiEndpoints.auth_login` (Django-allauth's headless API). On success: `setAuthenticated(true)` (re-runs `setApiDefaults()`), then `fetchUserState()` and `fetchGlobalStates(true)`. A 401 can mean an MFA challenge is pending, redirecting to `/mfa`. WebAuthn (`handleWebauthnLogin`, via `@github/webauthn-json`) and SSO (`ProviderLogin`, posting to `auth_provider_redirect`) are also supported.

6.3 Logout flow

`doLogout(navigate)` (`functions/auth.tsx`): calls `DELETE` on the session endpoint, clears the `mfa_trusted` and `csrftoken` cookies client-side, resets the Zustand user/server stores, navigates to `/login`.

6.4 Global state stores

Zustand (`create(...)`), not React Context, holds auth data (Context is reserved for dependency-injection-style providers: `ApiContext`, `ThemeContext`, `LanguageContext`):

- `states/UserState.tsx` — `user`, `is_authed`, `login_checked`, `isLoggedIn()`, role/permission checkers.
- `states/ServerApiState.tsx` — server info, allauth `authContext` / `mfaContext`.
- `states/LocalState.tsx` — persisted UI/session prefs.
- `states/states.tsx` — orchestrates `fetchGlobalStates()` after login.

`UserState` is **not persisted** — it's rebuilt every fresh page load by re-validating the session cookie, which is why `checkLoginState()` exists as the gateway page (`pages/Auth/LoggedIn.tsx`).

6.5 Protected routes

`components/nav/Layout.tsx` exports `ProtectedRoute`, which wraps the entire authenticated route subtree (structural gating — one parent route checks auth for all its children, not a per-route flag):

```
export const ProtectedRoute = ({ children }) => {
  const { isLoggedIn } = useUserState();
  if (!isLoggedIn()) return <Navigate to='/logged-in' state={{ redirectUrl: locati
  return children;
};
```

6.6 Permission gating

`states/UserState.tsx` exposes two families of checks — role-based (`hasViewRole` / `hasChangeRole` / `hasAddRole` / `hasDeleteRole`, keyed by `UserRoles` from `@lib/enums/Roles`) and model-permission-based (`hasViewPermission` / `hasChangePermission` / `hasAddPermission` / `hasDeletePermission`, keyed by `ModelType`). Both short-circuit `true` for superusers. Used everywhere to hide UI: row actions (§3.4), nav drawer entries, navbar tabs, settings menu items, and page edit-ability (`editEnabled={user.hasChangeRole(UserRoles.part)}`).

---

7. Navigation & Routing

7.1 Routing

`react-router-dom` v6. Route tree: **`src/router.tsx`**, mounted in a `BrowserRouter` inside `src/views/DesktopAppView.tsx`. Two sibling top-level `<Route path='/'>` blocks:

```
<Route path='/' element={<LayoutComponent />}>       {/* authenticated app shell *
  <Route index element={<Home />} />
  <Route path='part/'>
    <Route index element={<Navigate to='category/index/' />} />
    <Route path='category/:id?/*' element={<CategoryDetail />} />
    <Route path=':id/*' element={<PartDetail />} />
  </Route>
  {/* stock/, manufacturing/, purchasing/, sales/, core/... follow the same shape
</Route>

<Route path='/' element={<LoginLayoutComponent />}>   {/* unauthenticated shell */
  <Route path='/login' element={<Login />} />
  <Route path='/logout' element={<Logout />} />
  {/* register, mfa, mfa-setup, reset-password, verify-email/:key ... */}
</Route>
```

Each domain section follows the same pattern: an `index` route that `Navigate`s to a default sub-page, and a `:id/*` detail route (the trailing `/*` lets the detail page declare its own internal tabs without more `<Route>` nesting).

Layout routes (`components/nav/Layout.tsx` and `pages/Auth/Layout.tsx`) render an `<Outlet />` for their children — this is how Header/Footer/Nav wrap every authenticated page, and how the auth-flow shell wraps login/register/reset pages, without duplicating chrome per-page.

Lazy loading: nearly every page is `Loadable(lazy(() => import(...)))` (`functions/loading.tsx`), which wraps the lazy component in `Suspense`

(`functions/loading.tsx`), which wraps the lazy component in `Suspense`. `LayoutComponent`, `LoginLayoutComponent`, and `LoggedIn` instead use `EagerLoadable` (needed on every load; avoids `Suspense`'s commit-delay latency) and are explicitly preloaded once i18n is ready.

Protected routes: see §6.5 — `LayoutComponent` wraps its entire subtree in `ProtectedRoute`.

7.2 Navigation components (`src/components/nav/`)

- **Top navbar tabs**: config array in `src/defaults/links.tsx`, `getNavTabs(user)` → `{ame, title, icon, visible }` gated by role checks, rendered by `Header.tsx`'s `NavTabs()`.
- **Full nav drawer**: `NavigationDrawer.tsx` builds `MenuLinkItem[]` arrays (`{ id, title, link, icon, hidden }`), gated by `user.hasViewPermission/hasViewRole/isStaff`, rendered by the generic `components/items/MenuLinks.tsx`.
- **`NavigationTree.tsx`** — a separate, generic hierarchical drawer (Part categories / Stock locations) that lazily fetches children from the API, rather than a static config.
- **Breadcrumbs are manually built per page**, not derived from the route. Each detail page builds `{ name, url }[]` and passes it to `PageDetail` (`components/nav/PageDetail.tsx`), which appends an optional `lastCrumb` and hands the list to `BreadcrumbList.tsx` (truncates to first-3 + `...` + last-3).
- **Header**: `components/nav/Header.tsx` — logo/hamburger, nav tabs, global search, spotlight, barcode scan, notification bell (polls every 60s), user dropdown (`MainMenu.tsx`).

---

8. Styling

8.1 UI library

Mantine v9 across the board (`@mantine/core`, `dates`, `form`, `modals`, `notifications`, `spotlight`, `charts`, `carousel`, `dropzone`, `vanilla-extract`), plus `mantine-datatable` (tables) and `mantine-contextmenu` (right-click menus).

8.2 Theme

Two layers:

- `src/theme.ts` — a static empty `createTheme({})` whose sole purpose is generating CSS-variable bindings via `themeToVars()` (`@mantine/vanilla-extract`) for use in `.css.ts` files.
- `src/contexts/ThemeContext.tsx` — the real runtime theme, built dynamically from user-configurable settings in `LocalState` (`primaryColor`, `whiteColor`, `blackColor`,

`radius`, custom `breakpoints`) and passed into `<MantineProvider theme={...} colorSchemeManager={...}>`.

- **Dark/light mode**: Mantine's `colorSchemeManager`, custom `localStorageColorSchemeManager` (`lib/index.ts`), falling back to `prefers-color-scheme`. Components branch via `useMantineColorScheme()` directly when needed.

8.3 CSS approach — vanilla-extract, no CSS modules

All styling is `.css.ts` (no `.module.css` anywhere). Component-colocated style files sit next to their component (`ApiIcon.tsx` / `ApiIcon.css.ts`). The standard dark/light pattern:

```
export const layoutHeader = style({
  [vars.lightSelector]: { backgroundColor: vars.colors.gray[0] },
  [vars.darkSelector]: { backgroundColor: vars.colors.dark[6] },
});
```

Responsive rules use `'@media': { [vars.smallerThan('sm')]: {...} }`. Global overrides for third-party libraries that can't be targeted via vanilla-extract live in plain CSS: `src/styles/overrides.css`, imported once in `main.tsx`.

8.4 Icons

Tabler Icons (`@tabler/icons-react`) is the standard icon set — import `IconXxx` directly. `react-icons` is not used.

---

9. Project Conventions

- **File naming**: components/pages are PascalCase `.tsx`. Somewhat unusually, **hooks are also PascalCase filenames** (`UseForm.tsx`, `UseInstance.tsx`) even though the exported hook itself is camelCase (`useForm`). State/utility files also skew PascalCase (`LocalState.tsx`, `Api.tsx`). Style files share their component's base name (`ApiIcon.css.ts`).
- **Folder organization**: `tables/`, `forms/`, `pages/` are organized **by backend domain** (`part`, `stock`, `build`, `company`, `purchasing`, `sales`, `settings`...), mirroring Django app names. `components/` is organized **by UI concern** (`barcodes`, `buttons`, `dashboard`, `details`, `nav`, `panels`, `tables`...) — not by domain.
- **Path alias**: `@lib/*` → `./lib/*` (only alias defined in `tsconfig.json`). `tsconfig.lib.json` clears it when building the `lib/` package standalone.
- **Import order**: third-party imports first (alphabetized within braces), blank line, then relative/local imports, with `import type { ... }` for type-only imports. This is a convention, not an enforced Biome rule (no `organizeImports` config).

- **TypeScript**: mixed `interface` / `type`, with `type` more common; component props are typically an exported `type <Component>Props`. `strict: true`, but `noExplicitAny` is deliberately **off** in Biome — `any` does appear in real code, don't be afraid of it where genuinely needed but prefer concrete types where easy.
- **Linting/formatting**: **Biome** (repo-root `biome.json`), not ESLint. Single quotes, no trailing commas. Strict about unused imports (`noUnusedImports: "error"`) but relaxed on `any`, non-null assertions, array index keys, and exhaustive-deps compared to Biome's recommended defaults — match this pragmatic style rather than over-annotating.
- **`lib/` vs `src/`**: only put something in `lib/` if it's meant to be part of the public plugin SDK surface (`@inventreedb/ui`). Otherwise it belongs in `src/`.
- **i18n**: strings use Lingui (`t\`...\`` template tag). `yarn extract` / `yarn compile` manage translation catalogs — new user-facing strings must use `t\`` rather than raw literals.
- **No `useMutation`, no axios interceptors, no invalidateQueries-based cache busting** — these are deliberate absences (see §2.4, §2.5, §2.7); don't introduce them for a single new feature. Follow the existing plain-axios + `refreshTable()` / `refetch()` idiom instead.

---

10. Development Recipes (Quick Reference)

Call a new API endpoint

1. Add the route to `lib/enums/ApiEndpoints.tsx`.
2. Build URLs with `apiUrl(ApiEndpoints.x, pk?, pathParams?)` — never a literal string.
3. Reads → `useQuery` (reuse `useInstance()` for single objects); writes → `ApiForm`/modal hooks (§5.5) unless it's a non-form one-click action, in which case a direct `api.post/patch/delete(...)` is fine.
4. Handle errors with `showApiErrorMessage()` (reads) — writes via `ApiForm` handle this already.
5. Refresh with `table.refreshTable()` or the instance's `refetch()` / `refreshInstance()` — not `queryClient.invalidateQueries()`.

Create a new table

See §3.6. Files: `tables/<domain>/<Model>Table.tsx` (+ optional `<Model>TableFilters.tsx`). Component: `<InvenTreeTable url={...} tableState={...} columns={...} props={{...}} />`.

Create a new page

1. Add `pages/<domain>/<Name>.tsx`.
2. Register it in `router.tsx`: `export const Name = Loadable(lazy(() => import('./pages/<domain>/<Name>')));` then add a `<Route path='...' element={<Name />} />` under the correct layout/domain block.

{<Name />} /> under the correct layout/domain block.

3. If it needs breadcrumbs, build them locally and pass to `PageDetail` (§7.2).

Create a reusable component

- Domain-specific (tied to a model) → goes with that domain (`tables/` , `forms/` , `pages/`).
- Generic UI → `components/<concern>/` , following the existing concern folders — don't create a new top-level concern folder without a clear reason.
- Colocate a `.css.ts` file only if it needs custom styles beyond Mantine props.

Create a new form

See §5.7. Files: `forms/<Domain>Forms.tsx` (field set hook) + a modal hook call (`useCreateApiFormModal` / `useEditApiFormModal` / `useDeleteApiFormModal`) at the call site.

Create a modal

Don't build a raw `Modal` + manual submit logic for anything backed by an API model — use the modal hooks (§5.5), which already wrap `ApiForm` in a Mantine modal. For a non-form modal (e.g. a picker), use `src/hooks/UseModal.tsx` 's `useModal()` directly.

Add a new route

Add the page as above, register in `router.tsx` under the right `<Route path='...'>` block. Nest under an existing domain section if it belongs to a domain that already has one (e.g. new `part/` sub-page); the authenticated shell (`LayoutComponent`) auto-gates it via `ProtectedRoute` .

Add a new card

See §4.4. Detail-page field group → `DetailsField[]` + `<DetailsTable>` inside `<ItemDetailsGrid>` . Dashboard widget → register in `DashboardWidgetLibrary.tsx` .

Fetch data

`useQuery` with a `queryKey` covering every param that should trigger a refetch, `queryFn` doing `api.get(apiUrl(...), { params })` . Reuse `useInstance()` for single-object detail fetches.

Display data

Tables → `InvenTreeTable` + `TableColumn[]` . Detail fields → `DetailsTable` + `DetailsField[]` . Ad hoc → plain Mantine components (`Text` , `Badge` , `Group`), pulling values via `getValueAtPath(item, path)` if following the `DetailsTable` convention.

Handle loading states

Use the query's own `isLoading` / `isFetching` in JSX directly. For a component composing multiple queries, combine them into one derived boolean with a `useEffect` / `useMemo`, as `InvenTreeTable` does — don't add a redundant separate loading `useState` for a single query.

Handle errors

Reads: `throwOnError` + `showApiErrorMessage({ error, title })`. Writes: handled inside `ApiForm` automatically (field errors mapped via `form.setError`, non-field errors in a top `Alert`, non-400s via `invalidResponse()` / `showTimeoutNotification()`). Don't add axios interceptors — none exist by design; handle errors at the call site.

---

11. Common Mistakes to Avoid

- **Hardcoding API URL strings** instead of going through `ApiEndpoints` + `apiUrl()`.
- **Reaching for `useMutation`** — this codebase's mutation idiom is plain `api({...})` calls inside `ApiForm`, with manual `refreshTable()` / `refetch()` afterward.
- **Calling `queryClient.invalidateQueries()`** to refresh a table — bump `tableKey` via `table.refreshTable()` instead.
- **Building a custom table/grid from scratch** — always start from `InvenTreeTable` + `TableColumn[]`; every list-view feature (sort/filter/search/paginate/column-toggle) is already built in.
- **Writing a custom modal + manual submit for API-backed forms** — use `useCreateApiFormModal` / `useEditApiFormModal` / `useDeleteApiFormModal`.
- **Manually declaring full field metadata** (label, required, choices) in an `ApiFormFieldSet` when it's already derivable from the server's `OPTIONS` schema — only specify overrides.
- **Adding CSS Modules or plain `.css` files** for component styling — this project uses vanilla-extract (`.css.ts`) exclusively; plain CSS is reserved for third-party-library overrides only (`styles/overrides.css`).
- **Creating a new top-level `components/` concern folder** or a new domain folder in `tables/` `forms/` / `pages/` without checking whether an existing one already fits.
- **Skipping `t\`...\`` for user-facing strings** — breaks the Lingui i18n extraction workflow.

This guide reflects the frontend as of the exploration date. If a referenced file has moved or a pattern has changed, prefer what you observe in the current code over this document, and update the relevant section rather than treating it as fixed.